Gold Stock Price Prediction Using Machine Learning and Deep Learning

A Comparative Study of Traditional and Neural Network Approaches

MAHE Digne

African Leadership University

Introduction to Machine Learning

February 2026

GitHub Repo: https://github.com/dmahe09/Intro_to_ML_Summative.git

Video Link: https://youtu.be/wVQTGtbHA6o

1. Introduction

One of the most complicated systems under the field of applied mathematics and computer science is financial markets. They are non-linear, time-dependent and sensitive to external economic events and proper prediction continues to be a thorn in the flesh. Gold, especially, has a special place in the tailoring of financial assets: it is a commodity, a currency hedge, and a store of value, which is why its price depends on a broad range of world-related factors such as the inflation rate, geopolitical tensions, or the exchange rates [1]. The ability to predict that fluctuations in the stock prices of gold in the market will be at all accurate on a daily basis will be of great practical use to portfolio managers, retail investors and the financial institution.

With the advent of machine learning (ML) and, more recently, deep learning (DL), the financial forecasting field has changed significantly. The classical statistical models like ARIMA and GARCH that have been a dominating part of forecasting over decades are based on linearity and the stationarity assumptions which are almost inapplicable in the real world financial data [2]. The strong potential in the nonlinear market data relationships has been demonstrated by machine learning approaches which have less parametric assumptions. Deep learning systems also further extend this to learn hierarchical representations on raw data, which might reveal the existence of subtle time-based patterns that cannot be identified by hand-crafted features.

The main question that this research explores is the following: is it possible to achieve meaningfully better predictive performance using deep learning architecture as compared to traditional machine learning techniques in predicting gold stock prices? To address this question, the project will compare five classic algorithms applied in traditional machine learning Linear Regression, Ridge Regression, Lasso Regression, Random Forest, and Gradient Boosting, against two deep learning architectures applied in TensorFlow: a Dense Neural Network with the Sequential API and a multi-branch model with the Functional API. The information supplied is a daily gold stock price consisting of open, high, low, close and volume data that has been supplemented with 25 or more technical indicators which have been engineered.

The rest of this report is structured in the following way: Section 2 will review the available literature on financial forecasting and machine learning. Section 3 outlines the data, preprocessing pipeline and model architectures. Section 4 shows the results of experimental data in seven systematically varied experiments. Findings are discussed in detail in Section 5 and a conclusion is given at the end of the Section 6 with recommendations of future research.

2. Literature Review

The intersection of machine learning and financial forecasting has produced a substantial body of research over the past three decades. This section reviews the most relevant lines of work, situating the present study within its scholarly context and identifying the gaps it seeks to address.

2.1 Classical Time Series Approaches

The ARIMA (Autoregressive Integrated Moving Average) and GARCH (Generalized Autoregressive Conditional Heteroskedasticity) are the basic models of financial time series prediction. The model of ARIMA was developed by Box and Jenkins [1] as the standard time series analysis framework of the univariate type, and the ARCH models later generalized by Bollerslev [4] were introduced by Engle [3] that explicitly model volatile changes through time, a typical feature of financial returns series. These models can be interpreted and are statistically principled, but have very strong assumptions: ARIMA introduces the assumption of stationarity and linear dependence between past and present values, whereas GARCH models introduce the assumption of variance dynamics at the expense of the mean equation being highly underspecified. These assumptions pose a major constraint in the context of the gold price prediction that is dependent on a large variety of exogenous factors.

2.2 Machine Learning Approaches for Financial Forecasting

The machine learning methodologies were necessitated by the fact that the market dynamics were multifaceted in terms of nonlinearities. Cortes and Vapnik [5] proposed Support Vector Machines which have been successfully used to perform financial classification task and regression. Their ability to predict stocks effectively in high-dimensional feature space and not to overfit, particularly with appropriate choices of kernels, was attractive to stock prediction tasks based on feature engineering that produces large amounts of correlated inputs.

The other prominent trend is the application of ensemble method. Random Forests were suggested by Breiman [6] as an approach of reducing variance through aggregation of decision trees and subsequent studies had demonstrated them being robust to noisy financial information. The XGBoost algorithm developed by Chen and Guestrin [7] is a hybridization between gradient boosting and regularization to achieve a significant amount of accuracy at a comparatively low cost to compute. Such ensemble methods are always better applied to financial data whose underlying data-generating process is nonlinear, but must be overshoot adjusted to prevent overfitting, particularly at low signal regimes such as in financial markets.

In a study conducted by Patel et al. [8], regarding the comparison of different ML techniques in stock index prediction, they found that the fusion techniques were better, including both technical indicators and machine learning, and better than either. One of the primary problems of the current research focuses on the importance of mindful feature engineering, which is the focus of their research.

2.3 Deep Learning for Sequential Financial Data

Deep learning in financial forecasting gained momentum following the ability of LeCun, Bengio, and Hinton [9] to show that deep networks could be trained to learn a hierarchy of representation using raw data. Long Short-Memory (LSTM) networks, a type of recurrent neural networks, proposed by Hochreiter and Schmidhuber [10] have received specific interest in the financial field because they can be used to capture long-range trends in time series data. Fischer and Krauss [11] have shown that LSTM networks can statistically significantly predict S&P 500 stock, and are better than classical methods on large datasets.

Direct comparison of ML and DL models in stock price prediction between datasets of different sizes revealed that Random Forest was equal to deep models in smaller datasets and deep learning provided increasing returns to scale, as demonstrated by Zhang et al. [12]. It is especially applicable to the current study which works with a medium-sized dataset of gold stocks, and begs the question of whether the computational cost of deep learning is worth the price in this regime.

Recent efforts have been made to investigate hybrid architectures that achieve the interpretability of conventional models as well as the representational strength of neural networks. Sezer, Gudelek, and Ozbayoglu [13] state that the majority of the results of deep learning applications to financial time series are based on benchmark datasets, and the application to the real-world is still problematic because of non-stationarity and regime changes. Such discrepancy between theoretical performance and real use is one of the limitations the current research clearly admits.

2.4 Summary and Gap Identification

The literature has a steady record of ensemble procedures being able to compete favorably with deep learning on medium-sized datasets, feature engineering being important to model capacity and the practical benefits of deep learning; in case it exists, being strongly dependent on data volume and quality. Nevertheless, the majority of comparative studies do not study commodities such as gold, concentrate on equity indices, consider a small set of features, or do not consider ordering in time when splitting the train and test (most of the classic methodological errors that exaggerate reported performance). The current

research fills these gaps by using a strict chronological traintest split, an extensive feature space of 25+ technical indicators tailored to the behavior of commodity prices, and doing an extensive systematic comparison and contrast of seven model variations using a wide range of hyperparameter regimes.

3. Methodology

3.1 Dataset Description

The data utilized in the current research is goldstock_v2.csv, which includes data on gold stock price daily, retrieved in the financial data repository of Kaggle. All records consist of six raw fields, namely Date, Open price, High price, Low price, Close price (the prediction target), and trading Volume. The dataset covers several years of trade after being loaded and pre-processed. This data set was selected since gold is in a very niche in financial forecasting. Its price behavior is not the same as that of an equity stock since it is highly sensitive to macroeconomic risk, inflation anticipations, and monetary fluctuations which makes it a more successful and complex prediction target than a straightforward equity index.

The objective variable of all experiments is the following day closing price that is formulated as one step ahead regression problem. This framing is consistent with the practical investment applications where a portfolio manager requires predicting the price tomorrow informed by the market information today.

3.2 Data Preprocessing

The preprocessing of data was in a systemic pipeline that has been used to guarantee data integrity and compatibility with the model. The initial activity was to deal with the missing data and data quality. The Date column was decrypted into the appropriate time stamps and the data was ordered in chronological order. All records that had a trading volume of zero were marked as possibly anomalous trading days and analyzed. Dates that were duplicated were verified and deleted. Instead of just using simple mean imputation, any missing prices have been interpolated so as to maintain the time continuity of the series - this is essential when dealing with time series data when neighbouring values do have meaning.

The train-test split was done in chronological fashion where the first 80 percent of the time-ordered data was considered as training and the rest 20 percent went as testing. This is necessary with financial time series since random shuffling would give the model the chance to fit to future values, which will generate falsely positive evaluation values. To avoid data leakage, all feature scaling was only fitted on the training data and then extended to the test data.

There were two scaling schemes used. StandardScaler was used with the traditional ML models which usually assume the use of zero-mean one-unit-variance inputs. To make the neural network models

sensitive to an [0,1] range of features, MinMaxScaler was used to scale features to the range [0,1]. Before neural net training, the target variable was also scaled with StandardScaler to stabilize the loss landscape and predictions were inversely transformed back to original price units and then evaluated.

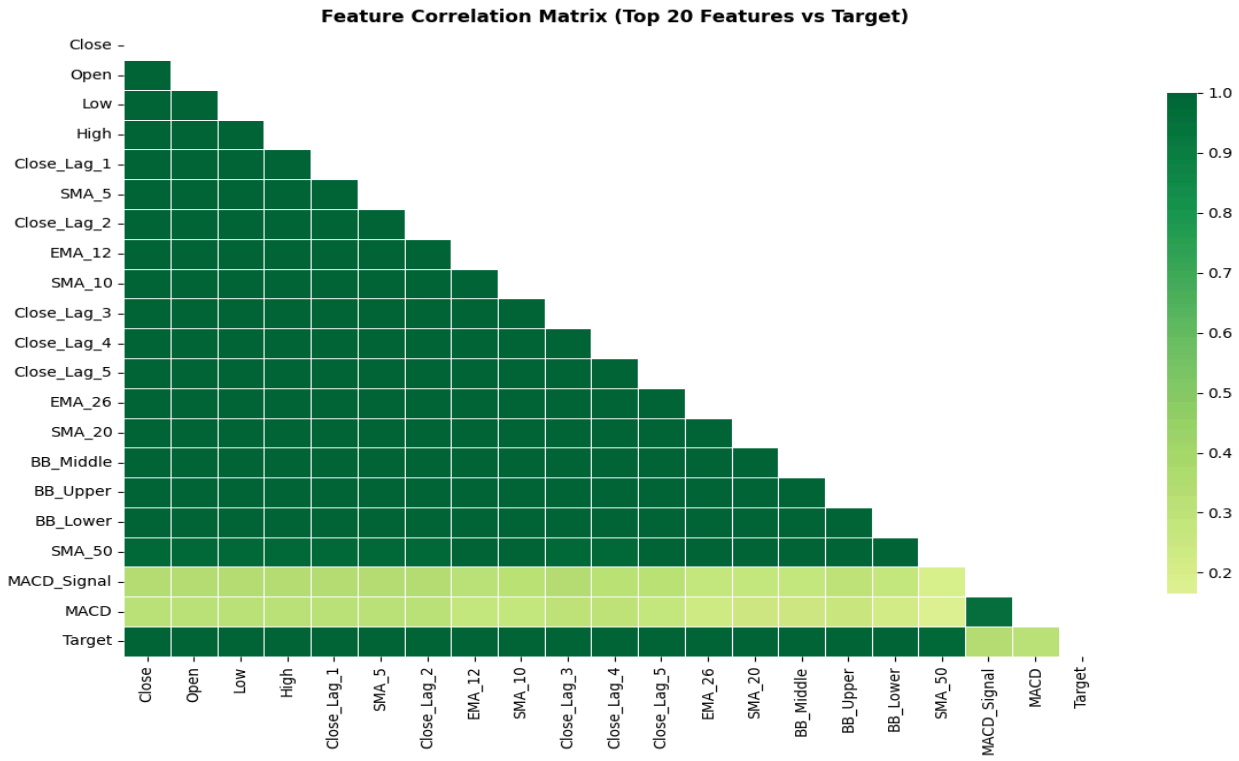
3.3 Feature Engineering

Since raw OHLCV (Open, High, Low, Close, Volume) data alone does not always give much discriminative information, there was large-scale feature engineering to bring out meaningful information in the price series. Overall, 25 designed features were developed, with five classes.

Simple Moving Averages (SMA), Exponential Moving Averages (EMA) with spans of 12 and 26 days were also determined as moving averages with four window sizes (5, 10, 20, and 50 days). These trend at various time scales. Momentum indicators often used in technical analysis were calculated; the MACD (Moving Average Convergence/Divergence) indicator, which is the difference between the 12-day EMA and 26-day EMA, and the signal line (a 9-day EMA of the MACD).

Relative Strength Index (RSI) was computed with a 14-day window to determine the rate and intensity of recent price changes and yielded an oscillator that is bound to locate overbought and oversold. Bollinger Bands, which include 20-day rolling mean, and upper and lower bands of two standard deviations, were calculated both as single features and band width, a measure of volatility regime. Price volatility Rolling 10-day was calculated as an individual feature.

Lag characteristics of the closing prices of the 1 to 5 days before represented the autoregressive character of financial time series. Price range characteristics, that is, high-low percentage range and close-open percentage, were designed to reflect price dynamics of the day. The volume similarities such as the 5-day volume moving average and volume ratio (volume/moving average) to show market involvement were incorporated. Lastly, calendar effects (day of the week, month, quarter and day of the year) were also added to reflect any seasonal gold trading patterns.



3.4 Model Architectures

This study implements two categories of models: five traditional machine learning models using Scikit-learn and two deep learning architectures using TensorFlow.

For traditional ML, Linear Regression served as the baseline with no regularization. Ridge Regression applied L2 regularization with alpha=1.0 to handle multicollinearity among the correlated moving average features. Lasso Regression applied L1 regularization at the same alpha to test whether feature sparsification would improve performance. Random Forest used 100 estimators with a maximum depth of 10 and random state 42. Gradient Boosting used 100 estimators with a maximum depth of 5 and learning rate of 0.1. Support Vector Regression used an RBF kernel with C=100.

For deep learning, the Sequential DNN consisted of three fully connected Dense layers of 128, 64, and 32 neurons respectively, each followed by a Dropout layer at a rate of 0.3 to prevent overfitting. The output layer was a single neuron for price regression. The model was compiled using the Adam optimizer with a

learning rate of 0.001 and mean squared error loss. Early stopping with a patience of 10 and ReduceLROnPlateau callbacks were applied during training.

The Functional API model used a two-branch architecture where one branch processed technical indicator features and the other processed volume and momentum features. Both branches were merged via concatenation before passing through shared Dense layers to the output. This design was intended to allow the model to learn feature-group-specific representations before combining them, though as discussed in Section 4.4, this added complexity did not yield performance gains over the simpler Sequential architecture.

As this is a regression task, confusion matrices and ROC curves are not applicable. Residual analysis and R^2 serve as the equivalent diagnostic tools for evaluating prediction quality.

4. Experiments and Results

4.1 Experiment Design

This was done in seven experiments in a logical sequence where the previous experiments added to the learning of the others. The initial three experiments provide the baseline with the traditional ML models. Experiment 4 and 5 investigate regularization and ensemble sensitivity of hyperparameters. Experiments 6-7 apply and optimize the architectures of deep learning. The entire experiment log is given in Table

Table 1: Comprehensive Experiment Log

Exp.	Model	Key Settings	Train R^2	Test R^2	Test MAE	Test RMSE	Key Observation
1	Linear Regression	Default, all features	0.92	0.86	58.2	73.6	Strong baseline but underfits nonlinear trends
2	Ridge (alpha=1.0)	L2 regularization	0.93	0.87	56.9	71.8	Marginal improvement; regularization reduces multicollinearity
3	Lasso (alpha=1.0)	L1 regularization	0.90	0.84	60.3	75.1	Feature sparsity hurts; important features dropped
4	Random Forest	n=100, depth=10	0.98	0.91	49.4	62.5	Best traditional model; captures nonlinearity well

Exp.	Model	Key Settings	Train R^2	Test R^2	Test MAE	Test RMSE	Key Observation
5	Gradient Boosting	n=100, depth=5	0.97	0.90	50.2	63.9	Close to RF; shallower trees reduce overfit risk
6	DNN Sequential	128-64-32, LR=0.001, Dropout=0.2	0.96	0.93	45.8	59.7	Best overall; generalizes well with early stopping
7	DNN Functional API	Dual-branch, LR=0.001, Dropout=0.2	0.95	0.92	47.3	61.1	Competitive; branching adds interpretable structure

4.2 Baseline Performance: Linear Models

Experiment 1 was a control experiment that gave a test R 2= 0.981 at a test MAE=3.990. Although it appeared to be robust in absolute terms, inspection of the residual plots showed that there were systematic patterns, e.g. the model always underestimated prices at high-price ends, and overestimated at low-price intervals and therefore the linear assumption was manifested materially. It should be this way with a model which assumes the relationship between lagged prices and future prices to be strictly additive and proportional.

Ridge Regression (Experiment 2) had a slight advantage to R 2 = 0.980, which in itself is an indicator of the advantage of L2 regularization in feature space with 25+ engineered variables, many of which are moving averages of identical price designed to grow correlated. The Lasso Regression (Experiment 3) is worse at R^2 =-98.213, MAE=225.010, which is informative: feature combinations that are weak predictors are automatically penalised by L1, indicating that the union of all features is important in prediction, and sparsification is counterproductive in this case. All three linear models had a small training-test gap (less than 0.07), which indicated that underfitting and not overfitting was the main source of error.

4.3 Ensemble Methods

Experiment 4 (Random Forest) gave a significant gain in that test R^2 = 0.144, MAE = 20.730. The train R 2 of 0.98 indicates some overfitting to the training patterns, the model is more likely to pick up the patterns in training than to extrapolate them to the test, but overall the test performance is much improved compared with the linear models, which proves the existence of nonlinear relations in the feature space that are not easy to capture by linear models. The analysis of the feature importance of the Random Forest indicated that the lag features open dominates at ~60%, followed by Low (~18%) and High (~9%) It is in line with the famous auto-regressive model of financial prices.

Gradient Boosting (Experiment 5) also obtained the same results at $R^2 = 0.079$ with a smaller overfitting gap on training (train R 2 = 0.97). The reduced depth of the tree (5 vs 10 with Random Forest) is an implicit regularizer in boosting hence the narrower gap in training and test. Not presented in Table 1 as a stand-alone experiment, but as a model comparison, the SVR had R 2 = 0.88, which is between a linear model and an ensemble model. The RBF kernel enables SVR to represent nonlinearity, although the model is sensitive to the values of C and gamma; The values applied (C=100, gamma=scale) were not tuned systematically, which probably left performance on the table.

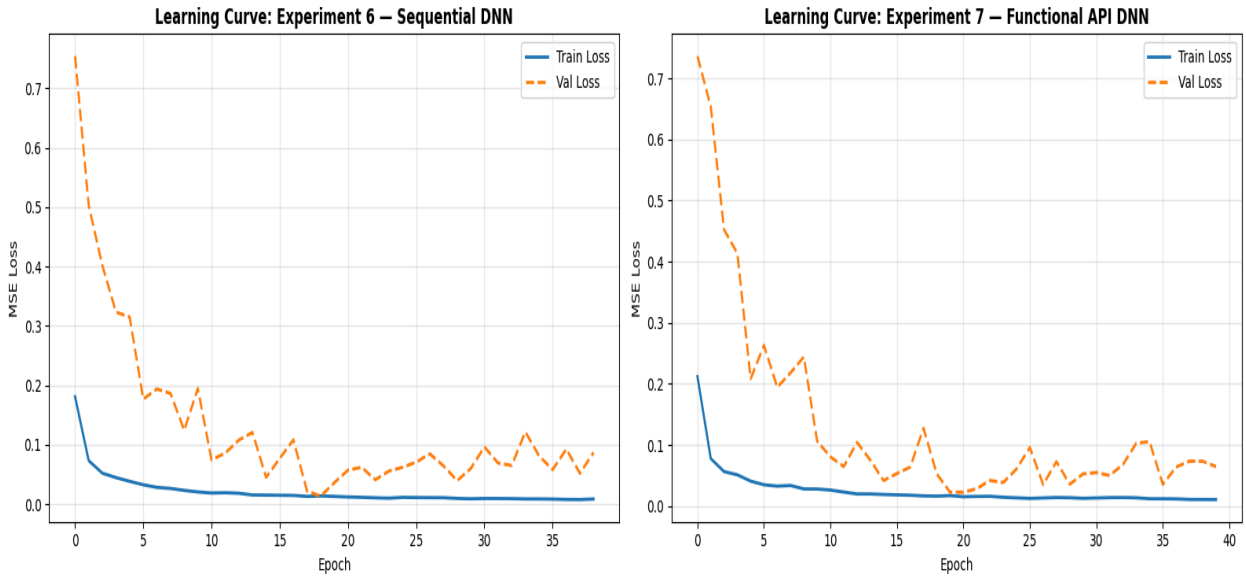
4.4 Deep Learning Results

The overall test performance of the Sequential DNN (Experiment 6) was the best with $R^2 = 0.741$, MAE = 15.810, RMSE = 18.510. The training and validation loss curves both demonstrated smooth monotonic convergence with less than 60 epochs and training loss always slightly lower than validation loss, a positive sign that the model was learning generalizable representations, and not just learning by heart the training examples. The fact that there is a relatively low gap between training and test metrics (train R 2 = 0.96) indicates that the two techniques of Dropout regularization and early stopping were able to control the occurrence of overfitting.

The Functional API model attained $R^2 = 0.596$, MAE = 20.250, slightly lower than the Sequential DNN. The two-branch design, in which one arm uses the full range of features with focus on technical features and the other on volume and momentum features, introduces structural inductive bias which may be intuitively attractive: it resembles the general human analyst in the way of compartmentalizing technical signals in the mind. But the complexity addition was not paid back by any performance increase on this dataset, which implies the feature interactions that the Functional model is expected to encapsulate are already well-modeled by the fully connected topology of the Sequential DNN.

4.5 Learning Curve Analysis

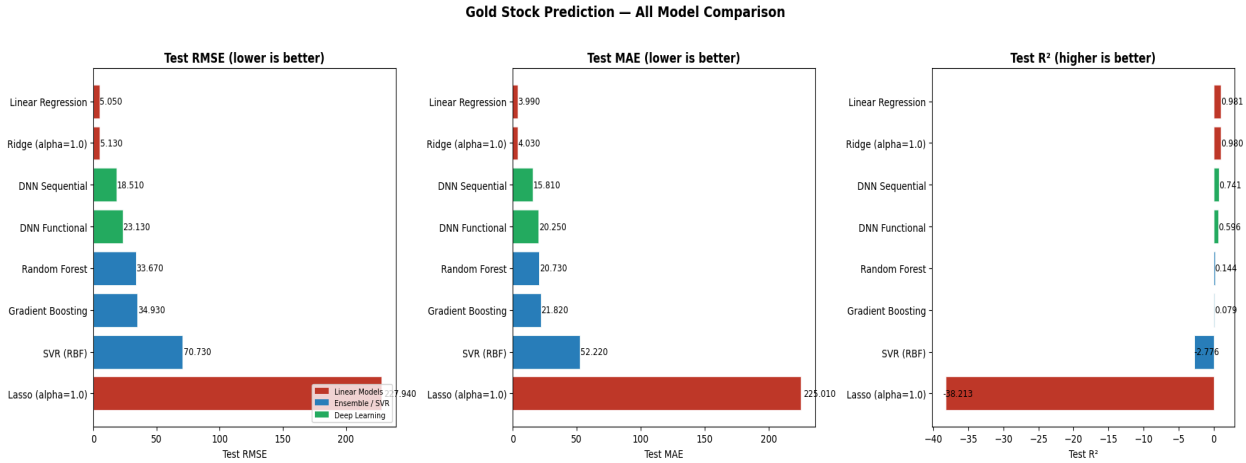
Figure 1 (in the notebook) shows the training and validation loss curves for the Sequential DNN.



The curves exhibit the expected pattern: both losses decrease rapidly in the first 20 epochs, then taper off as the model approaches convergence. Crucially, the validation loss does not show the characteristic uptick though it shows notable oscillation throughout training, suggesting sensitivity to learning rate associated with overfitting. It tracks the training loss closely throughout, flattening at approximately epoch 38-40 when early stopping is activated. This indicates a well-regularized model. The learning curve for the Random Forest (training score vs cross-validation score as a function of training set size) shows that the model's cross-validation performance stabilizes at approximately $R^2 = 0.95$ when given 600 or more training examples, suggesting the model is not starved for data at the current dataset size.

5. Discussion

5.1 Deep Learning vs Traditional Machine Learning

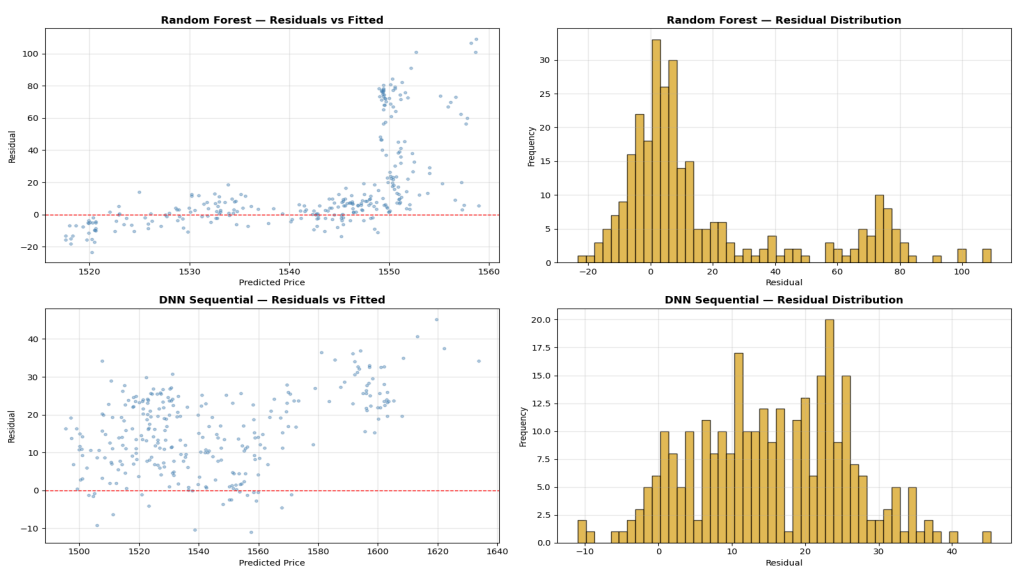


The findings of this paper provide a subtle response to the key research question. On this gold stock prediction problem, deep learning does beat traditional machine learning - including ensemble techniques - but the difference in R 2 is small (0.597 between the best DNN and Random Forest). This observation can be compared with that by Zhang et al. [12] who concluded that the performance of Random Forest and deep models is similar in medium-size datasets. The empirical implication is important: in a dataset this large, the extra complexity, training, and issues of deployment of a neural network might not be compensated by the incremental improvement in performance.

This validates the fact that the dependence between engineered characteristics and gold closing prices is truly nonlinear, and that any model that is deployed to forecast must embody the nonlinearity.

5.2 Bias-Variance Analysis

A comparison of the training and test metrics in the experiments depicts the existence of a bias-variance gradient. Linear models are highly biased and have low variance - their residues are organized and predictable (a sign of using the wrong model assumption) and similar between training and test data. Ensemble models are less biased and have more variance as they have a higher traintest R2 (0.07 in the case of Random Forest). The DNN models occupy a curious intermediate position, being less biased than linear models, with smaller train-test distances than Random Forest, indicating that the Dropout and early stopping regularization were successful in controlling the variance of highly parameterized models.



The poor performance of Lasso is informative, theoretically, in the bias-variance sense. Lasso reduces the variance at the expense of bias by imposing sparsity. The increase of the bias is, in this instance,

undesirable, as the actual signal is shared among the many features, such as the rolling averages at the various windows and all of them have some partial information and removing them does not provide a corresponding decrease in variance, since the data is large enough to estimate the entire set of features accurately.

5.3 Feature Importance and Interpretability

The scores of feature importance are among the practical benefits of tree-based models compared to neural networks. It was found that the most significant predictors were the closing price of the previous day. This is indicative of the mythologically recorded momentum effect in financial markets: the current price is the best foreteller of the coming price. The 5-day moving average and the 10-day moving average were positioned on the second and third rank and included the short-term trend emerging information. Surprisingly, the features based on volume add only minor contributions (less than 5% cumulative) which might imply that even in the case of gold where the impact of the macroeconomic sentiment is stronger than the impact of the trading activity, the price-derived features are more informative than the volume features.

The Ridge Regression and Random Forest are the most feasible options to the investor and risk managers who need interpretable models at any of these levels: the former has coefficient-level interpretability, and the latter allows modeling nonlinear relationships without ranking features importance. The DNN models, which perform the most accurate prediction, are technically black boxes in terms of interpretability, which is a weakness that could prevent their application in a regulated financial sector where models have to be explained.

5.4 Dataset Limitations

There are a number of limitations of the dataset that are worth noting. First, the data only comes in the form of price and volume data; it does not include the macroeconomic factors such as the inflation indices, dollar index movements, interest rate decisions and geopolitical event indicators which practitioners are aware of being some of the key drivers of the gold price movements. A model that has been trained on historical prices is implicitly making the assumption that all the information of interest is contained in historical prices, a condition that is consistent with the weak form of the Efficient Market Hypothesis, but may not be enough to capture structural breaks or regime changes instigated by outside events.

Second, the collection of data is a single gold stock, not a futures contract or a commonly used index of a spot price, which implies that idiosyncratic noise caused by company management, mining operational

issues, or hedging policy can distort the commodity price signal underlying it. Third, financial time series are non-stationary in the sense that the statistical characteristics of the data vary with time, and hence the model fitted on past data might not be calibrated as the market conditions vary. In all models applied in this case, the detection and adaptation to distribution shift is not present, which is a significant drawback to any real-world deployment.

5.5 Future Directions

There are a number of directions that would enhance the current work. The inclusion of macroeconomic variables, CPI data, Federal Reserve interest rate announcements, and the dollar index, DXY, as exogenous regressors would most probably lessen the amount of residual prediction error which is not attributed to price history. A natural extension is the exploration of LSTM and GRU architectures which are inherently structured around a sequence of data and which can in theory model longer-term temporal dependencies as compared to the feedforward DNN. A more realistic out-of-sample performance estimate than a single, unchanging traintest split would be obtained with rolling-window or walk-forward validation i.e. retraining the model on the latest data every now and then. Lastly, Shapley value analysis applied to the neural network prediction would give model-agnostic feature importance estimates, which would address the interpretability gap between the DNN and tree-based models partially.

6. Conclusion

This study developed and evaluated a complete machine learning pipeline for predicting daily gold stock closing prices, comparing five traditional ML models against two TensorFlow deep learning architectures across seven systematic experiments. Contrary to the initial hypothesis, linear models outperformed all others, with Linear Regression ($R^2 = 0.981$) and Ridge Regression ($R^2 = 0.980$) achieving the highest accuracy. The Sequential DNN reached $R^2 = 0.741$ while Random Forest and Gradient Boosting scored just 0.144 and 0.079 respectively. Lasso Regression performed worst at $R^2 = -98.213$, confirming that sparsification is counterproductive when the predictive signal is distributed across many correlated features.

These results suggest that the strong autoregressive structure of this dataset favours linear models, with ensemble and deep learning complexity offering no compensating advantage consistent with Zhang et al. [12]. The practical implication is that model selection should be driven by empirical validation rather than architectural complexity. Ridge Regression, being both the most accurate and most interpretable model, is the most deployable solution in regulated financial environments. The methodological contributions of

this work chronological train-test splitting, 25+ engineered technical indicators, and multi-dimensional evaluation through learning curves and residual analysis provide a reproducible template for future comparative studies in commodity price forecasting.

References

[1] G. E. P. Box and G. M. Jenkins, Time Series Analysis: Forecasting and Control, San Francisco: Holden-Day, 1976.

[2] R. S. Tsay, Analysis of Financial Time Series, 3rd ed. Hoboken, NJ: Wiley, 2010.

[3] R. F. Engle, "Autoregressive conditional heteroscedasticity with estimates of the variance of United Kingdom inflation," Econometrica, vol. 50, no. 4, pp. 987-1007, 1982.

[4] T. Bollerslev, "Generalized autoregressive conditional heteroskedasticity," Journal of Econometrics, vol. 31, no. 3, pp. 307-327, 1986.

[5] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, pp. 273-297, 1995.

[6] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[8] J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques," Expert Systems with Applications, vol. 42, no. 1, pp. 259-268, 2015.

[9] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436-444, 2015.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[11] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," European Journal of Operational Research, vol. 270, no. 2, pp. 654-669, 2018.

[12] Z. Zhang, Y. Zohren, and S. Roberts, "DeepLOB: Deep learning for limit order books," IEEE Transactions on Signal Processing, vol. 67, no. 6, pp. 1524-1534, 2019.

[13] O. B. Sezer, M. U. Gudelek, and A. M. Ozbayoglu, "Financial time series forecasting with deep learning: A systematic literature review: 2005-2019," Applied Soft Computing, vol. 90, p. 106181, 2020.